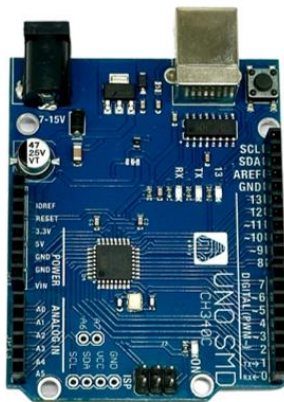


Comparison Between Arduino Abstraction and Bare-Metal AVR C Programming Through a Multiplexed 7-Segment Display

Carlos Hernández Hevia

11th May 2026



COPYRIGHT USE

Copyright and License Notice

© 2026, Carlos Hernández Hevia. All rights reserved.

This document, including all text, diagrams, data, designs, images, and any other content contained herein, is the exclusive intellectual property of the author named above.

Usage Terms

Academic Use: Permission is granted for academic, educational, or non-commercial research purposes, provided that proper citation is given. You may quote, reference, and distribute limited excerpts for these purposes, as long as you clearly attribute the work to the original author.

AI Training & Machine Learning: You are strictly prohibited from using any part of this document, in whole or in part, to train, fine-tune, or otherwise develop any artificial intelligence (AI), machine learning, or large language model (LLM), including but not limited to models such as ChatGPT, Claude, Llama, Gemini, and similar systems.

Commercial & Profit Use: You are strictly prohibited from using this document for any commercial, profit-generating, advertising, or revenue-generating purpose, including but not limited to selling, licensing, or incorporating this work into a paid product or service, without explicit prior written consent from the author(s).

License

This work is licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0).

To view a copy of this license, visit:

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

Additional Clarification: The "NoDerivatives" (ND) clause explicitly prohibits the creation of derivative works. For the purposes of this document, training an AI model on this content constitutes the creation of a derivative work and is therefore expressly forbidden.



Contents

Summary	4
Connections	4
Initial programming tests	6
Counter using Arduino programming language.....	8
Counter with C programming.....	9
Script settings.....	11
Defining and writing digital output ports	12
C script for 3461BS-1 multiplexed 7-segment display	12
Compiling and sending the program to the board in Linux.....	14
Conclusions and observations	16
References.....	17

Summary

The objective of this project is to get started with low-level embedded programming through a practical personal project. The work focuses on developing basic digital input/output control for an Arduino UNO board in order to operate a 3461BS-1 4-digit 7-segment LED display.

The main goal is to implement a counter for the display using both Arduino language and raw C. To achieve this, the project first defines the objectives, required electronic components and circuit configuration before moving on to the software implementation. The first version of the counter is developed using Arduino IDE and the Arduino programming language to verify the correct operation of the display and the multiplexing logic. Once the system works correctly, the same functionality is implemented again using direct C programming and register manipulation. This second approach aims to provide a better understanding of low-level programming concepts, including memory registers, digital I/O control and hardware interaction on the ATmega328P microcontroller.

Connections

First, I will make the electronic circuit necessary for controlling the 3461BS-1. According to its datasheet [1], the display pins are:

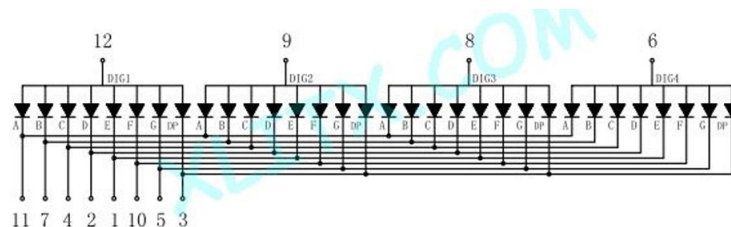


Figure 1: 3461BS-1 Schematic [1]

This component is a multiplexed 7 segment display (actually 8 considering the point). This means that in every instant only one of the four digits is switched on, but by changing rapidly between digits the human eye sees a constant number. This reduces the number of pins from 32 to 12. So, we will need 12 digital output signals.

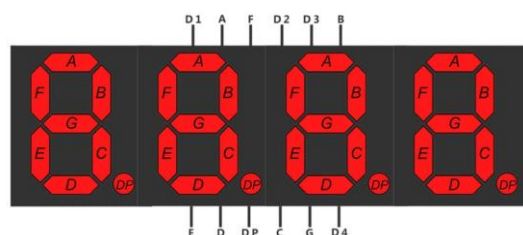


Figure 2: 3461BS-1 pinout [2]

Protoboard Line	Pin Letter	Display Pin Number	Arduino Pin Number
20	D1	12	12
21	A	11	11
22	F	10	10
23	D2	9	9
24	D3	8	8
25	B	7	7
40	E	1	13
39	D	2	2
38	DP	3	3
37	C	4	4
36	G	5	5
35	D4	6	6

In order to protect the device, I will put a small 220K resistor before pins 12, 9, 8 and 6 to limit current. Even though Arduino UNO already does this, adding a resistor provides an extra security layer with low voltage drop.

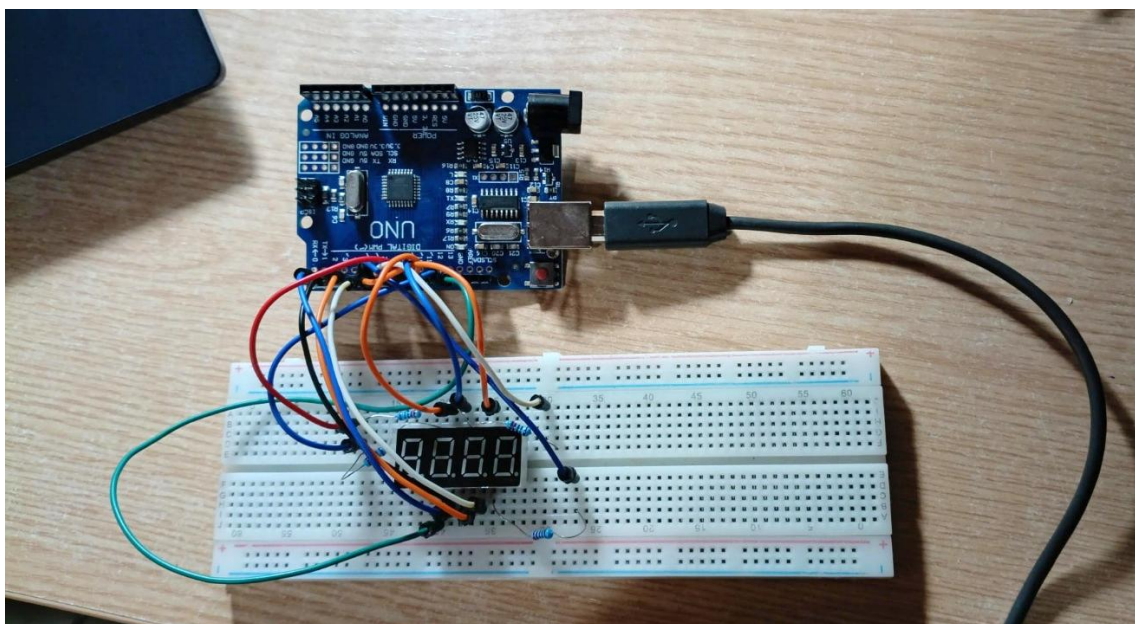


Figure 3: connections between the display and the Arduino, using a protoboard.

Initial programming tests

Once everything was connected, I made a simple test to see if everything worked properly. I made a simple program in Arduino in which all the segments of a display would turn on for a second. Then, the next digit is tested and so on.

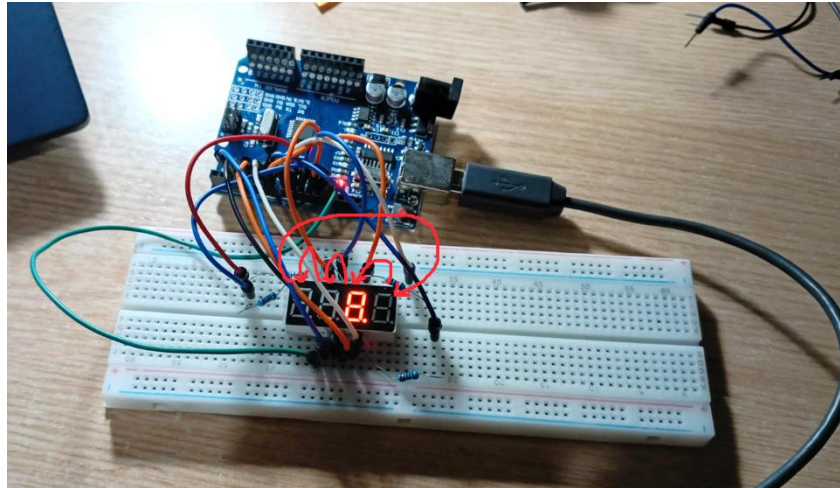


Figure 4: Simple arduino checking that connections are all ok

```
void setup() {  
  // put your setup code here, to run once:  
  int i = 0;  
  for(i=2; i<=13; i++){  
    pinMode(i, OUTPUT);  
  }  
}  
  
void loop() {  
  // put your main code here, to run repeatedly:  
  int k;  
  int digits[]={6, 8, 9, 12};  
  for(k=0; k<4; k++){  
    digitalWrite(digits[k], HIGH);  
    delay(500);  
    digitalWrite(digits[k], LOW);  
    delay(500); //Si no ponemos dos, pasa inmediatamente al estado anterior  
  }  
}
```

Since all the LEDs worked well, we can move on. The next thing that we should do is making a function that is called every time a number and certain digit is written. This will make the code more modular and easier in the future.

(Note: There are plenty of libraries and tools to make this with little effort. However, my goal with this is to reinforce concepts and improve my understanding of this systems).

Function for writing numbers in the desired digit

```
void MuestraNumero(int digito, int
numero){
    //Seleccionar dígito:
    digitalWrite(D4, LOW);
    digitalWrite(D3, LOW);
    digitalWrite(D2, LOW);
    digitalWrite(D1, LOW);
    digitalWrite(digito, HIGH);

    /* AHORA ES LÓGICA INVERSA:
       0/LOW = segmento ON
       1/HIGH = segmento OFF*/
    digitalWrite(D, HIGH);
    digitalWrite(DP, HIGH);
    digitalWrite(C, HIGH);
    digitalWrite(G, HIGH);
    digitalWrite(B, HIGH);
    digitalWrite(F, HIGH);
    digitalWrite(A, HIGH);
    digitalWrite(E, HIGH);

    //Empezamos a hacer casos
    switch(numero){
        case 0:
            digitalWrite(D, LOW);
            digitalWrite(C, LOW);
            digitalWrite(B, LOW);
            digitalWrite(F, LOW);
            digitalWrite(A, LOW);
            digitalWrite(E, LOW);
            break;
        case 1:
            digitalWrite(C, LOW);
            digitalWrite(B, LOW);
            break;
        case 2:
            digitalWrite(A, LOW);
            digitalWrite(B, LOW);
            digitalWrite(D, LOW);
            digitalWrite(E, LOW);
            digitalWrite(G, LOW);
            break;
        case 3:
            digitalWrite(A, LOW);
            digitalWrite(B, LOW);
            digitalWrite(C, LOW);
            digitalWrite(D, LOW);
            digitalWrite(G, LOW);
            break;
        case 4:
            digitalWrite(B, LOW);
            digitalWrite(C, LOW);
            digitalWrite(F, LOW);
            digitalWrite(G, LOW);
            break;
        case 5:
```

```
            digitalWrite(A, LOW);
            digitalWrite(C, LOW);
            digitalWrite(D, LOW);
            digitalWrite(F, LOW);
            digitalWrite(G, LOW);
            break;
        case 6:
            digitalWrite(D, LOW);
            digitalWrite(C, LOW);
            digitalWrite(G, LOW);
            digitalWrite(F, LOW);
            digitalWrite(A, LOW);
            digitalWrite(E, LOW);
            break;
        case 7:
            digitalWrite(A, LOW);
            digitalWrite(B, LOW);
            digitalWrite(C, LOW);
            break;
        case 8:
            digitalWrite(A, LOW);
            digitalWrite(B, LOW);
            digitalWrite(C, LOW);
            digitalWrite(D, LOW);
            digitalWrite(E, LOW);
            digitalWrite(F, LOW);
            digitalWrite(G, LOW);
            break;
        case 9:
            digitalWrite(A, LOW);
            digitalWrite(B, LOW);
            digitalWrite(C, LOW);
            digitalWrite(F, LOW);
            digitalWrite(G, LOW);
            break;
        default:
            break;
    }
    return;
}
```


Counter using Arduino programming language

After everything we've covered, we're finally ready to build the counter using Arduino. The idea is simple: count from 0 to 9.999, then roll back to 0. But we can't just light up one digit at a time with a long delay: that would look like a single digit is on. To make all four digits appear lit at the same time, we need to multiplex: switch between digits rapidly, each one for just a few milliseconds. That way, persistence of vision makes them look steady. That means `delay(10000)` is useless here. Instead, we use short, repeated delays to refresh the display constantly. I implemented this with two levels of control: the main `loop()` function, and a `while` loop that uses `millis()`. `millis()` tracks the time since the board powered up, so we can handle timing without blocking the code. This lets the display keep refreshing while the counter advances at the right pace.

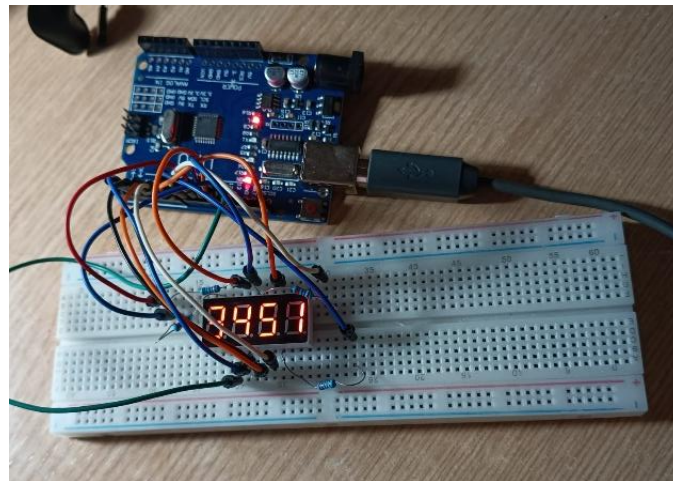


Figure 5: Display showing the four digits at once (5ms transition)

```
// Variables globales de inicialización:

//Cambiar según el conexionado con los pines de Arduino:
int A=11; int B=7; int C=4;
int D=2; int E=13; int F=10;
int G=5; int DP=3; int D1=12;
int D2=9; int D3=8; int D4=6;
int k;

int cuenta[] = {0,0,0,0};
int t_numero = 0.01 * 1000; //Segundos que permanece encendido cada número
(ojo con el formato)
int t_cambio_digito = 5; // ms que tarda en cambiar de digito

void setup() {
  // Activar todas las pines como salidas digitales:
  for(k=2; k<=13; k++){
    pinMode(k, OUTPUT);
  }
}

void loop() {
  unsigned long comienzo = millis(); // Millis da los ms desde el arranque

  while(millis() - comienzo < t_numero){

    MuestraNumero(D1,cuenta[0]);
    delay(t_cambio_digito);
    MuestraNumero(D2,cuenta[1]);
    delay(t_cambio_digito);
    MuestraNumero(D3,cuenta[2]);
    delay(t_cambio_digito);
    MuestraNumero(D4,cuenta[3]);
    delay(t_cambio_digito);
    //t_transicion = t_transicion + t_cambio_digito*4;
  }
}
```


Counter with C programming

In the second part of this exercise, the objective was to perform the same task previously implemented in Arduino language but using raw C instead. The purpose of this approach was to gain a better understanding of low-level programming and how the microcontroller interacts directly with the hardware.

To begin with, it was necessary to understand what Arduino functions such as `digitalWrite()` actually do internally. These functions are part of a Hardware Abstraction Layer (HAL), which simplifies hardware control by hiding the direct manipulation of memory registers. By programming directly in C, this abstraction layer is removed, allowing direct access to the ATmega328P registers and providing a clearer understanding of how the microcontroller operates.

Memory registers are specific memory locations inside the CPU that are permanently connected to hardware peripherals and pins. Unlike general-purpose RAM, these registers are used to configure and control the behavior of the microcontroller, including digital inputs, outputs and communication with external components. Working directly with these registers helped to better understand concepts such as bit manipulation, pin configuration and low-level hardware control.

Below there is an image showing part of the ATmega328P register map.

13.4 Register Description

13.4.2 PORTB – The Port B Data Register

Bit	7	6	5	4	3	2	1	0	
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0	PORTB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.3 DDRB – The Port B Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x04 (0x24)	DDRB7	DDRB6	DDRB5	DDRB4	DDRB3	DDRB2	DDRB1	DDRB0	DDRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.4 PINB – The Port B Input Pins Address

Bit	7	6	5	4	3	2	1	0	
0x03 (0x23)	PINB7	PINB6	PINB5	PINB4	PINB3	PINB2	PINB1	PINB0	PINB
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

13.4.8 PORTD – The Port D Data Register

Bit	7	6	5	4	3	2	1	0	
0x0B (0x2B)	PORTD7	PORTD6	PORTD5	PORTD4	PORTD3	PORTD2	PORTD1	PORTD0	PORTD
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.9 DDRD – The Port D Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x0A (0x2A)	DDRD7	DDRD6	DDRD5	DDRD4	DDRD3	DDRD2	DDRD1	DDRD0	DDRD
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.10 PIND – The Port D Input Pins Address

Bit	7	6	5	4	3	2	1	0	
0x09 (0x29)	PIND7	PIND6	PIND5	PIND4	PIND3	PIND2	PIND1	PIND0	PIND
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

Figure 6: AT Mega 328P registers [3].

Registers are faster than RAM since they are inside the CPU but are fixed. RAM is general purpose on the other hand. In figure 6, we obtained from the CPU datasheet the registers of the digital pins:

- PORTB is a 1 Byte register that specifies the output voltage, which can be 1 (High, 5V) or 0 (Low, 0V).
- DDRB is a 1 Byte register that specifies whether the pins are inputs or outputs.
- PINB is a 1 Byte register used for reading when pins act like digital inputs.

To know which bits correspond to which bits we can see the following figure in the datasheet:

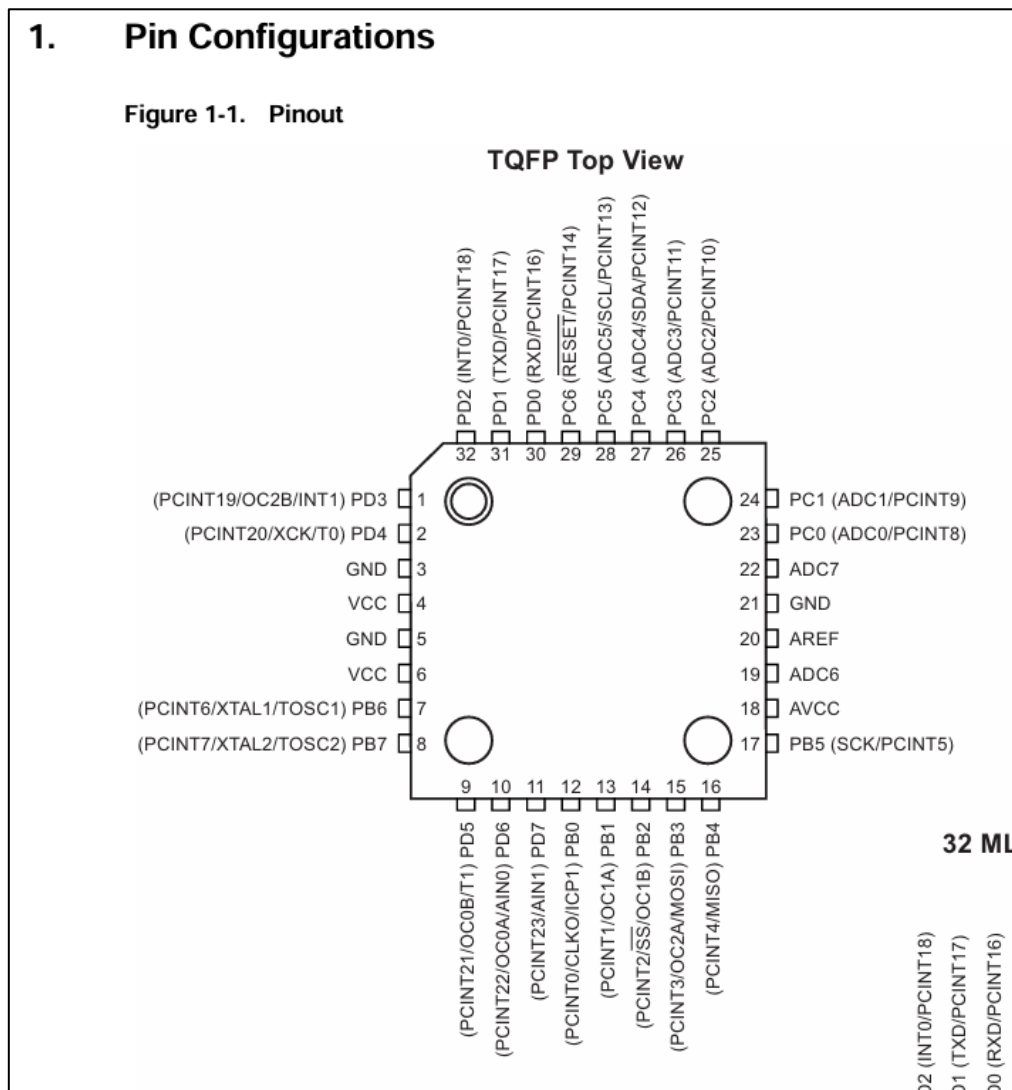


Figure 7: AT mega328P pinout diagram [3]

If we summarise it in a table:

Arduino Pin	AVR Port & Bit	Register bit to control	Alternate functions (not used in this task)
RX	PD0	PIND0, DDRD0, PORTD0	Serial communication (receive)
TX	PD1	PIND1, DDRD1, PORTD1	Serial communication (transmit)

2	PD2	PIND2, DDRD2, PORTD2	External interrupt 0 (INT0)
3	PD3	PIND3, DDRD3, PORTD3	External interrupt 1 (INT1), PWM (OC2B)
4	PD4	PIND4, DDRD4, PORTD4	XCK (USART clock), T0 (Timer 0 clock input)
5	PD5	PIND5, DDRD5, PORTD5	PWM (OC0B)
6	PD6	PIND6, DDRD6, PORTD6	PWM (OC0A)
7	PD7	PIND7, DDRD7, PORTD7	–
8	PB0	PINB0, DDRB0, PORTB0	PCINT0 (Pin Change Interrupt 0)
9	PB1	PINB1, DDRB1, PORTB1	PWM (OC1A)
10	PB2	PINB2, DDRB2, PORTB2	PWM (OC1B), SS (SPI Slave Select)
11	PB3	PINB3, DDRB3, PORTB3	PWM (OC2A), MOSI (SPI)
12	PB4	PINB4, DDRB4, PORTB4	MISO (SPI)
13	PB5	PINB5, DDRB5, PORTB5	SCK (SPI clock), built-in LED

Script settings

Once we know the registers and their pinout, we can start to code in C. First of all, we will need the following libraries and definitions:

Definition / Library	Explanation
<code>#define F_CPU 16000000UL</code>	Define CPU frequency (16MHz) as long unsigned for functions which will use it.
<code>#include <avr/io.h></code>	Loads all the hardware variables of the AVR microcontrollers family. This defines (DDRB, PORTB and so on).
<code>#include <util/delay.h></code>	Loads the <code>_delay_ms()</code> function
<code>#include <stdbool.h></code>	Allows the usage of Boolean variables

For this exercise, the `_delay_ms()` function available in C was used to control the timing of the program. This function pauses the execution for a specified number of milliseconds, making it useful for simple timing operations and easy to implement in small projects. Although this approach is sufficient for this exercise, it is not considered the most efficient or scalable solution for time management in embedded systems.

In more advanced applications, hardware timers are generally preferred because they allow more precise and reliable timing control without blocking the execution of the program. Timers and watchdog modules are commonly used in embedded systems to manage periodic events, synchronization and system stability. For simplicity purposes, and because this project does not require high performance or multitasking capabilities, the use of `_delay_ms()` was considered adequate.

To compile and upload the .c source files, the development environment was moved to a Linux operating system. The programs were compiled and executed directly from the terminal using the AVR toolchain, which provided a better understanding of the compilation and uploading process compared to the automated workflow of the Arduino IDE.

Defining and writing digital output ports

For setting digital output ports, we will use the following syntax:

```
DDRD |= (1 << PD2);
```

This statement means:

- DDRD: the byte which defines whether the pins are output or input digital signals.
- “|= “ is C OR operator. When combined with (1<<PD2) it means that for this register, for the bit corresponding to pin 2, it will do an OR comparison and will set that bit at HIGH, meaning that pin 2 will be a digital output.
 - In this exercise, all the digital pins are outputs, but if we wanted to set a pin as input, we would write the following:
 - ```
DDRD &= (0 << PD2);
```
    - This is an AND operation, setting the PD2 bit in DDRD register at LOW, making it a digital input signal.

For establishing the output value, we will use two programming approaches:

1. Setting all the outputs at once:

```
PORTD &= 0b00000011;
```

This makes pins 0 to 7 (corresponding to PORTD bits PD2, PD3 ... PD7) to be forced into LOW state. RX and TX are not modified since the AND operator with a logic 1 does not interfere with their previous state. The same principle applies for PB0, PB1 ... PB5 (pins 8 to 13):

```
PORTB &= 0b11000000;
```

2. Setting each bit ON/OFF separately:
  - a. For setting a HIGH state:

```
PORTD |= (1 << PD3); //For pin 3
```

- a. For setting a LOW state:

```
PORTB &= (1 << PB0); //For pin 8
```

## C script for 3461BS-1 multiplexed 7-segment display

The final code used for the counter is relatively similar to the one in Arduino, just adding the low-level statements required.

```

#define F_CPU 16000000UL //CPU
frequency, 16MHz

#include <avr/io.h> // AVR hardware
variables
#include <util/delay.h> //_delay_ms()
#include <stdbool.h> // booleans

int digitalWrite_mia(int pin, bool
bit);
void MuestraNumero(int digito, int
numero);

// Variables globales:
int A=11; int B=7; int C=4;
int D=2; int E=13; int F=10;
int G=5; int DP=3; int D1=12;
int D2=9; int D3=8; int D4=6;
int k;

int main(int argc, char *argv[]){
//Iniciación de variables
// Set as outputs
// Puerto D → D2 a D7
DDRD |= (1 << PD2);
DDRD |= (1 << PD3);
DDRD |= (1 << PD4);
DDRD |= (1 << PD5);
DDRD |= (1 << PD6);
DDRD |= (1 << PD7);
// Puerto B → D8 a D13
DDRB |= (1 << PB0);
DDRB |= (1 << PB1);
DDRB |= (1 << PB2);
DDRB |= (1 << PB3);
DDRB |= (1 << PB4);
DDRB |= (1 << PB5);

// Poner todos en LOW:
// D0-D7 LOW
PORTD &= 0b00000011;
// D8-D13 LOW
PORTB &= 0b11000000;

unsigned t_cambio_digito = 5;
unsigned t_cambio_numero = 500;
int cuenta[] = {0,0,0,0};
unsigned t_actual_ciclo = 0;

while(1){//Rutina de ejecución
t_actual_ciclo = 0;
while(t_actual_ciclo <
t_cambio_numero){
MuestraNumero(D1,cuenta[0]);
_delay_ms(t_cambio_digito);
MuestraNumero(D2,cuenta[1]);
_delay_ms(t_cambio_digito);
MuestraNumero(D3,cuenta[2]);
_delay_ms(t_cambio_digito);
MuestraNumero(D4,cuenta[3]);
_delay_ms(t_cambio_digito);
}
}
}

```

```

t_actual_ciclo =
t_actual_ciclo + t_cambio_digito*4;
}
// Count up
cuenta[3]++;
if(cuenta[3]==10){
cuenta[3]=0;
cuenta[2]++;
}
if(cuenta[2]==10){
cuenta[2]=0;
cuenta[1]++;
}
if(cuenta[1]==10){
cuenta[1]=0;
cuenta[0]++;
}
if(cuenta[0]==10){
cuenta[0] = 0;
cuenta[1] = 0;
cuenta[2] = 0;
cuenta[3] = 0;
}
}
return 0;
}

int digitalWrite_mia(int pin, bool
bit)
{
// Los pines 0-7 PORTD
if(pin >= 0 && pin <= 7)
{
if(bit) //Poner en HIGH
PORTD |= (1 << pin);
else // Poner en LOW
PORTD &= ~(1 << pin);
}
else if(pin >= 8 && pin <= 13) //
Los pines 8-13 pertenecen a PORTB
{ // PB0==8 → offset:
pin -= 8;
if(bit)
PORTB |= (1 << pin);
else
PORTB &= ~(1 << pin);
}
else
{
return -1;
}
return 0;
}
}

```

The *MuestraNumero()* has been omitted, but the function is just like the one made in Arduino but calling *digitalWrite\_mia()*.

## Compiling and sending the program to the board in Linux

After finishing the C script, we are now able to compile and send the program to the board. The first thing that we need to do is download the compiler and libraries that we need:

***sudo apt update***

***sudo apt install gcc-avr avr-libc avrdude***

This install, in order: the C compiler for the AVR microcontroller family, the AVR library for C and the “avrdude” to send the .hex file to the board.

Next, we compile the C script and generate an .elf file. Executable Linkable Format files are Linux executables with all the information of the compiled C file [4].

***avr-gcc -mmcu=atmega328p -Os Counter-3461BS-1.c -o Counter-3461BS-1.elf***

In this command, we specify:

- a. That we use the AVR C compiler.
- b. The microcontroller model.
- c. Os: optimizes size for less storage size in the board’s memory.
- d. The C file we want to compile.
- e. The .elf file we get.

After generating the .elf, we must generate the .hex file, which is the file sent to the microcontroller. This is a lighter file with hexadecimal instructions [5].

***avr-objcopy -O ihex -R .eeprom Counter-3461BS-1.elf Counter-3461BS-1.hex***

Here, we are telling the terminal:

- a. To use AVR to copy object files to other formats.
- b. -O ihex: specify output format (Intel hexadecimal).
- c. -R eeprom: removes the EEPROM section of the file to avoid overwriting.
- d. ELF file name as input.
- e. HEX file name as output.

This .hex is what we want to send to the board. To do so, we have to find the USB port in which the board is connected. We can look for the port using:

***ls /dev/tty\****

This command looks for USB/UART ports (tty\*) in the devices folder. We found the “dev/ttyUSB0” which is very likely the board.

Finally, we can send the .hex file to the board with the following command:

***avrdude -c arduino -p m328p -P /dev/ttyUSB0 -b 115200 -U flash:w: Counter-3461BS-1.hex:i***



The information here is:

- a. AVRdude is a free software tool useful for sending our program to the board from the terminal, without the ArduinoIDE.
- b. -c Arduino: this is the bootloader (Arduino in this case).
- c. -p m328p: microcontroller code.
- d. -P /dev/USB0: USB port in which the board is connected.
- e. -b 115200: baud rate, for our device it is 115.200 Bd.
- f. -U flash:w: name.hex:i : specifies to write (w) the hex file in the flash memory. The hex:i indicates that the file is a Intel hexadecimal.

After all of this, the board works and functions properly.

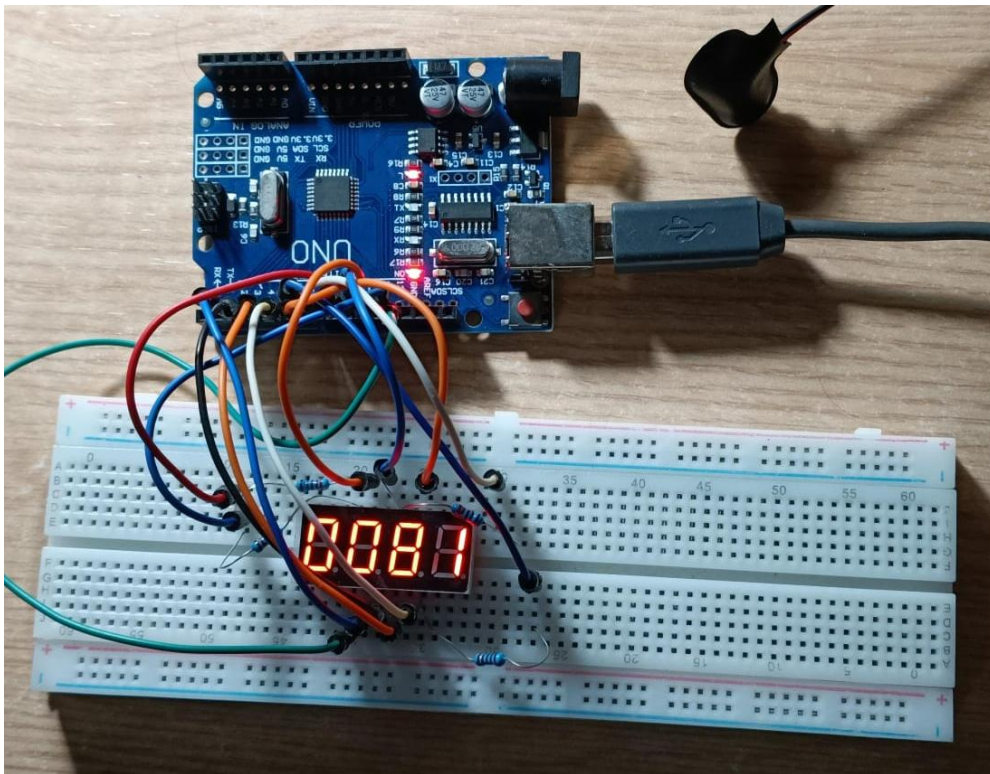


Figure 8: display running programmed in C.

## Conclusions and observations

This project allowed me to understand the basic operation of a microcontroller at both a high and low level. Through the development of a counter using a multiplexed 4-digit 7-segment display, I learned how memory registers work and how digital inputs and outputs are controlled directly on the microcontroller. Working with multiplexing also helped me better understand timing, signal control and the behavior of shared display lines.

Another important part of the exercise was implementing the same project in both Arduino and raw C. This made it possible to compare the abstraction layers used in each approach and understand the advantages and disadvantages of both. Programming in Arduino IDE simplified the interaction with the hardware through functions such as `digitalWrite()`, while programming in C required direct manipulation of registers, increasing complexity but also providing greater control and efficiency.

A remarkable observation was that the ghosting effect on the display segments was noticeably lower when programming in C compared to Arduino. This is likely due to the faster and more deterministic execution of direct register operations.

As a possible improvement, the display refresh loop could be controlled using hardware timers instead of delay-based loops, allowing more precise timing and better overall performance.

# References

- [1] XLITX, “3461BS-1 display Datasheet,” [Online]. Available:  
<https://www.xlitx.com/datasheet/3461BS.pdf>.
- [2] D. Kokkinos, “Software Particles,” 29 October 2023. [Online]. Available:  
<https://softwareparticles.com/learn-how-a-4-digit-7-segment-led-display-works-and-how-to-control-it-using-an-arduino/>.
- [3] Atmel, “ATmega328P Datasheet,” [Online]. Available:  
[https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P\\_Datasheet.pdf](https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf).
- [4] Wikimedia Foundation, “ELF file - Wikipedia,” [Online]. Available:  
[https://es.wikipedia.org/wiki/Executable\\_and\\_Linkable\\_Format](https://es.wikipedia.org/wiki/Executable_and_Linkable_Format).
- [5] Wikimedia Foundation, “HEX file - Wikipedia,” [Online]. Available:  
[https://es.wikipedia.org/wiki/HEX\\_\(Intel\)](https://es.wikipedia.org/wiki/HEX_(Intel)).